Speech Commands Unknown-Handling: Final Experimental Report

Michal Iwaniuk, Bartlomiej Borycki

May 4, 2026

Abstract

This report studies unknown-word handling in small-footprint keyword spotting on the Speech Commands v0.01 dataset. The task is formulated as a 12-class problem with 10 target commands, an aggregated `unknown` class, and a synthesized `silence` class. We compare four unknown-handling strategies across three neural backbones: BC-ResNet, Keyword Transformer (KWT), and MatchboxNet. The experimental protocol contains three stages: a backbone-strategy comparison, a $2 \times 2 \times 2$ augmentation study for the Stage 1 winners, and a reduced-depth KWT follow-up.

The results show that Strategy A, which trains on the original 30 spoken-word labels plus `silence` and remaps non-target predictions to `unknown` at inference time, achieves the best validation macro-F1 for all three backbones. Augmentation effects are architecture-dependent: BC-ResNet is already near saturation, MatchboxNet benefits mainly from time shift and SpecAugment, and KWT benefits most from enabling all three augmentation factors. The best validation model achieved validation macro-F1 = 0.967, test macro-F1 = 0.964, and test accuracy = 0.978. The reduced-depth KWT follow-up shows that time-only and time-frequency attention variants remain competitive, but frequency-only attention is insufficient.

1 Description of the research problem

The research problem is to build and evaluate keyword-spotting models that classify 1-second, 16 kHz speech clips from the Speech Commands v0.01 corpus. The original dataset contains 30 spoken-word folders and a `_background_noise_` folder with long ambient recordings. The final evaluation does not use all spoken words as separate classes. Instead, it reduces the dataset to a 12-class recognition problem:

- **Target commands**: `yes`, `no`, `up`, `down`, `left`, `right`, `on`, `off`, `stop`, and `go`;
- **Special classes**: `unknown`, which groups the remaining 20 spoken words, and `silence`, which is synthesized from cropped background-noise recordings.

The official `validation_list.txt` and `testing_list.txt` files define the validation and test partitions, so the experiments preserve the standard Speech Commands split protocol.

The most distinctive part of the research concerns how to represent the `unknown` class. The study compares four strategies, described in Section 3, each corresponding to a different interpretation of the same 12-class evaluation problem.

Under this task definition, the central research question is how the representation of the `unknown` class affects performance across different neural architectures. The problem is harder than ordinary closed-set classification for two reasons. First, `unknown` is semantically heterogeneous: many different non-target words must be rejected without being confused with the ten target commands. Second, the data are imbalanced, because the aggregated `unknown` pool is much larger than any individual keyword class. The prepared dataset statistics shown in Table 1 make that imbalance explicit: `unknown` accounts for 32,550 of the 52,942 training examples.

Table 1: Split summary derived from the prepared dataset metadata

Split	Total	Keywords	Unknown	Silence
Train	52,942	18,538	32,550	1,854
Validation	7,056	2,577	4,221	258
Test	7,092	2,567	4,268	257

2 Theoretical introduction and literature review

2.1 Keyword spotting as an open-set recognition problem

Keyword spotting (KWS) systems are designed to detect a small set of predefined spoken commands from a continuous audio stream, while rejecting all other speech, non-speech sounds, and silence. Formally, let $\mathcal{K} = \{c_1, c_2, \ldots, c_K\}$ denote the set of K target keywords. At inference time, the model must decide whether an input audio segment belongs to one of the target classes or should be mapped to a *reject* class, typically termed `unknown`. This is an instance of *open-set recognition* [10], where the space of possible inputs is unbounded, and the training set covers only a finite subset of known classes. A successful KWS model must therefore learn a representation that provides well-separated decision regions for the target commands while leaving a large, clearly defined region for anomalies.

2.2 Neural architectures for KWS

Below we review the key architectural characteristics of the three model families used in this study.

2.2.1 Self-attention and the Keyword Transformer

The Transformer architecture [6] avoids recurrence and convolution, using instead multi-head self-attention to capture global dependencies. Given an input sequence $\mathbf{X} \in \mathbb{R}^{L \times d_{\text{model}}}$, the self-attention operation for a single head is defined in Equation (1):

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}, \tag{1}$$

with queries $\mathbf{Q} = \mathbf{X}\mathbf{W}^Q$, keys $\mathbf{K} = \mathbf{X}\mathbf{W}^K$, and values $\mathbf{V} = \mathbf{X}\mathbf{W}^V$. Multiple heads run in parallel, and their outputs are concatenated and projected. Positional information is injected via learned or fixed positional encodings.

The Keyword Transformer (KWT) [7] adapts this paradigm to KWS by applying self-attention directly along the time axis of the input spectrogram. Let $\mathbf{S} \in \mathbb{R}^{T \times F}$ be the spectrogram with T time frames and F frequency bins. A single linear projection $\mathbf{W}_0 \in \mathbb{R}^{F \times d}$ maps each time frame to a d-dimensional token, so the input is treated as a sequence of length T in which each token is one time window with all frequency bins as its features. A learnable classification token is prepended and a learned positional embedding is added, after which the resulting sequence of length $T + 1$ is processed by a stack of multi-head self-attention encoder blocks; the final state of the classification token is fed to a linear classifier. Because attention spans the full temporal extent at every layer, KWT can model long-range phonetic patterns natively, which is advantageous when the keyword position is not centrally aligned in the input window. The cost is a quadratic complexity $\mathcal{O}(L^2)$ in the sequence length, although for short KWS sequences this remains manageable. The KWT authors note that, empirically, attention is more effective in the time domain than in the frequency domain—a finding that motivates the time/frequency/both ablation in our Stage 3.

2.2.2 Depthwise-separable convolutions and broadcasted residuals

Standard 2D convolutions mix spatial and channel correlations with a kernel of size $F \times T \times C_{\text{in}} \times C_{\text{out}}$, which leads to a high computational cost. Depthwise-separable convolutions [4] factor this operation into a spatial depthwise convolution applied separately to each input channel, followed by a pointwise 1×1 convolution that mixes channels, as in Equation (2):

$$\text{DSConv}(\mathbf{X}) = \text{Pointwise}(\text{Depthwise}(\mathbf{X}; \mathbf{W}_d);\ \mathbf{W}_p), \tag{2}$$

where $\mathbf{W}_d \in \mathbb{R}^{F \times T \times C_{\text{in}}}$ and $\mathbf{W}_p \in \mathbb{R}^{1 \times 1 \times C_{\text{in}} \times C_{\text{out}}}$. The compute cost relative to a standard convolution is approximately $\frac{1}{C_{\text{out}}} + \frac{1}{F \cdot T}$, which in typical KWS settings yields close to an order-of-magnitude reduction in both parameters and FLOPs because spatial mixing and channel mixing are decoupled.

Residual connections [3] ease optimization by letting layers learn the residual form in Equation (3):

$$\mathbf{Y} = \mathcal{F}(\mathbf{X}) + \mathbf{X}. \tag{3}$$

Combining depthwise-separable convolutions with skip connections results in the TC-ResNet family [9]. BC-ResNet [5] extends this idea through *broadcasted residual learning*, which combines cheap 1D temporal computations with a small amount of 2D frequency-aware processing. The residual function is decomposed into a 2D part $\mathcal{F}_2$ (frequency-depthwise convolution with sub-spectral normalisation) and a 1D temporal part $\mathcal{F}_1$ (depthwise temporal convolution followed by pointwise channel mixing). The 2D output is averaged over the frequency axis to obtain a temporal feature, $\mathcal{F}_1$ operates on this temporal feature, and the result is then broadcast back along the frequency axis before being added to the identity shortcut. Writing avgpool_F for averaging over frequency and BC for the broadcast back to the original frequency-time shape, the BC-ResBlock takes the form shown in Equation (4):

$$\mathbf{Y} = \mathbf{X} + \mathcal{F}_2(\mathbf{X}) + \text{BC}(\mathcal{F}_1(\text{avgpool}_F(\mathcal{F}_2(\mathbf{X})))). \tag{4}$$

The heavy pointwise operations inside $\mathcal{F}_1$ run on a tensor whose frequency dimension has been collapsed to one, reducing their cost by a factor proportional to the number of frequency bins while preserving the inductive bias of 2D processing.

2.2.3 Time-channel separable 1D convolutions

Inspired by efficient ASR architectures, MatchboxNet [8] employs purely 1D convolutions along the time axis using time-channel separable (TCS) blocks. A TCS sub-block first applies a depthwise 1D convolution along the time dimension, which processes each channel independently, and then a pointwise 1×1 convolution that mixes channels, followed by batch normalisation, ReLU, and dropout, as formalized in Equation (5):

$$\text{TCS}(\mathbf{X}) = \text{Dropout}\Big(\text{ReLU}(\text{BN}(\text{Pointwise}(\text{Conv}_{1D}(\mathbf{X};\ \mathbf{W}_{\text{time}});\ \mathbf{W}_p)))\Big), \tag{5}$$

where Conv_{1D} is a depthwise 1D convolution along time with kernel size K and $\mathbf{W}_p$ is the pointwise mixing matrix. A MatchboxNet block stacks several TCS sub-blocks with a residual connection. This design separates temporal mixing (cheap depthwise 1D convolutions) from channel mixing (cheap pointwise convolutions), dramatically reducing parameters while maintaining high discriminative power.

2.3 Strategies for representing the unknown class

The experiments compare four ways of representing the `unknown` class. Strategy A trains on all original word labels and remaps non-target predictions to `unknown` at inference time.

Strategy B collapses all non-target words into one `unknown` label before training. Strategy C uses a hierarchical coarse/fine classifier, while Strategy D combines known-class training with confidence-based rejection and outlier exposure [11]. The exact training targets, inference rules, and sampling regimes are described with the conducted experiments in Section 3.

2.4 Data augmentation for robust keyword spotting

Short, cleanly segmented speech samples make KWS models susceptible to overfitting. Data augmentation introduces controlled variations that reflect real-world acoustic conditions, which improves generalization. In our experiments, whenever a given augmentation factor was enabled, the following parameter settings were used consistently across the corresponding runs.

Time shifting. Each non-`silence` training waveform is translated by a random offset $\Delta \sim \mathcal{U}(-T_{\max}, T_{\max})$. In the conducted experiments, $T_{\max} = 100\,\text{ms}$, which corresponds to ± 1600 samples at the 16 kHz sampling rate. Samples shifted outside the analysis window are discarded, and the newly revealed region is padded with zeros according to Equation (6):

$$x_{\text{shift}}[n] = \begin{cases} x[n-\Delta] & \text{for } 0 \le n - \Delta < N, \\ 0 & \text{otherwise.} \end{cases} \tag{6}$$

This simulates the temporal uncertainty of a streaming keyword spotter and is one of the earliest and most effective augmentations for KWS [1, 2].

Background noise mixing. Real environmental noise $n(t)$ is added to the speech signal $x(t)$ at a gain $\alpha \sim \mathcal{U}(0, \alpha_{\max})$. In the experiments, background mixing is attempted for each non-`silence` training example with probability $p = 0.8$, using $\alpha_{\max} = 0.3$, as shown in Equation (7):

$$x_{\text{mix}} = \text{clip}(x + \alpha\, n, -1, 1), \tag{7}$$

where n is drawn from the prepared training `silence` pool. Clipping ensures the waveform stays within the valid amplitude range. This augmentation forces the model to focus on robust phonetic cues rather than relying on silence detection.

SpecAugment. Instead of manipulating the raw waveform, SpecAugment [12] applies spectral-domain masking directly to the input feature map $\mathbf{X} \in \mathbb{R}^{F \times T}$. A time mask zeroes out t_{mask} consecutive time frames, and a frequency mask zeroes out f_{mask} consecutive frequency bins, with mask widths chosen uniformly at random. Equation (8) shows the frequency-mask case:

$$\begin{aligned} &\mathbf{X}_{f,t} \leftarrow 0 \\ &\qquad \text{for } f \in [f_0, f_0 + f_{\text{mask}}),\ t \in [0, T). \end{aligned} \tag{8}$$

and analogously for time masks. In the experiments, SpecAugment is applied after frontend feature extraction and normalization. Each enabled sample receives one time mask with width sampled from 0 to 10 frames and one frequency mask with width sampled from 0 to 5 bins; zero-width masks are skipped. This acts as a strong regulariser similar to structured dropout, forcing the network to use distributed features across both time and frequency dimensions.

These three augmentation families are often combined; their individual contributions and interactions are the subject of the controlled $2 \times 2 \times 2$ factorial experiment described in Section 3.

Table 2: Backbone-specific settings for the executed runs

Backbone	Frontend	Optimizer	Batch	Epochs	Key model settings
BC-ResNet	40-bin log-Mel	AdamW, $lr = 0.01$	512	100	variant `bcresnet-1`, base channels 8, stage blocks 2/2/4/4, dropout 0.1
KWT	40-bin log-Mel	AdamW, $lr = 10^{-3}$	512	100	`kwt-1`, $d_model = 64$, depth 12, heads 1, MLP dim 256, cosine schedule with 5-epoch warmup
MatchboxNet	64-dim MFCC	NovoGrad, $lr = 0.05$	128	100	`matchboxnet-3x1x64`, block channels 64, kernels 13/15/17, conv kernels 11 and 29, dropout 0.1

3 Description of the conducted experiments

The executed study follows the three-stage protocol defined in the project plan. It contains 42 completed training runs: 12 in Stage 1, 24 in Stage 2, and 6 in Stage 3. Model selection was based on validation 12-class macro-F1.

All runs used the official train/validation/test split files. BC-ResNet and KWT used the shared 40-bin log-Mel frontend, while MatchboxNet used its native 64-dimensional MFCC frontend. The backbone-specific training settings are summarized in Table 2.

Stage 1: backbone and unknown-handling comparison

Stage 1 used a minimal shared augmentation baseline consisting of random time shift and background-noise mixing, with SpecAugment disabled.

Stage 1 trained each of the three model families from scratch under four unknown-handling strategies:

Strategy A - post-hoc remapping Train on all 31 (30 commands + `silence`) original labels, then remap any non-target prediction to `unknown` at inference time.

Strategy B - collapsed training labels Collapse all 20 non-target words into a single `unknown` label before training and train a 12-class classifier directly. This is the most standard and task-aligned protocol, and it is the setup followed by BC-ResNet, KWT, and MatchboxNet for this particular task [5, 7, 8].

To prevent the `unknown` category from dominating training, non-target word classes are rebalanced so that the total number of `unknown` examples is approximately equal to the average size of the target keyword classes. Examples are sampled in a class-aware manner, drawing a similar number of samples from each non-target word class to preserve diversity within the `unknown` category and avoid over-representation of a few frequent classes.

Strategy C - two-stage classification A hierarchical two-head architecture is used. A shared backbone feeds two classification heads. Head 1 performs 3-way classification into `silence`, `unknown`, and `keyword`. Head 2 performs 10-way classification over the target keywords. During training, the loss of Head 1 is computed for all samples using standard cross-entropy with the corresponding 3-class target. The loss of Head 2 is computed only for samples whose ground-truth label belongs to the 10 target keywords; for `silence` and `unknown` samples, the Head 2 loss is

masked out and does not contribute to optimization. The total loss is given by Equation (9):

$$\mathcal{L} = \mathcal{L}_{\text{head1}} + \lambda\, \mathbf{1}[y \in \mathcal{K}]\, \mathcal{L}_{\text{head2}}, \tag{9}$$

where $\mathcal{K}$ denotes the set of 10 target keywords, and λ is a scalar weight controlling the contribution of the second head ($\lambda = 1$ is used in experiments). During inference, the prediction of Head 1 is used first; Head 2 is consulted only when Head 1 predicts `keyword`.

For the hierarchical 3-way head (`silence`/`unknown`/`keyword`), class imbalance is handled by balanced sampling minibatches so that silence, unknown, and keyword are sampled in roughly equal proportions.

Strategy D - confidence-based rejection with uniform targets This strategy is inspired by *Deep Anomaly Detection with Outlier Exposure* [11].

Train only an 11-class classifier for the known classes (`10 commands + silence`). For non-target training words, encourage a uniform output distribution over the 11 known classes (i.e., the target label for `unknown` class becomes $[1/11, \ldots, 1/11]$). Then, classify a sample as `unknown` at inference time when Equation (10) holds:

$$\max_k p(y = k \mid x) < \tau. \tag{10}$$

The threshold τ is selected based on the highest f1-macro score on the validation set. For fairness, the non-target word pool uses the same class-aware balancing policy as in Strategy B, so that outlier exposure remains diverse without overwhelming the target keyword classes.

Sampling differences The four unknown-handling strategies were not just label transforms; they also changed the effective sampling regime. Table 3 combines the sampler implementation with the prepared dataset counts and shows why the strategies are not directly comparable in terms of exposure per epoch. Strategy A sees the natural training distribution, Strategies B and D downsample through a weighted sampler with replacement, and Strategy C oversamples the three superclasses by cycling through class buffers to build balanced batches.

Table 3: Strategy mechanics and effective examples seen per epoch

Strategy	Training target	Sampler behavior	Examples per epoch
A	original 30 spoken labels + `silence`	plain shuffled pass over the full train set	52,942
B	direct 12-class training with collapsed `unknown`	weighted sampler with replacement and class-aware unknown balancing	22,246
C	hierarchical 3-way head + 10-way keyword head	superclass-balanced batch sampler with cyclic reuse of class buffers	97,792 for BC-ResNet/KWT, 97,664 for MatchboxNet
D	11-class training + confidence-based rejection	same weighted sampler as Strategy B	22,246

Stage 2: augmentation grid

Stage 2 kept only the best Stage 1 unknown-handling strategy for each backbone and evaluated it on the full $2 \times 2 \times 2$ augmentation grid, where each of the three augmentation factors was either disabled or enabled.

Stage 3: reduced-depth KWT follow-up

Stage 3 reused the best KWT configuration selected from Stage 2 and varied only the transformer attention setup. The follow-up kept `depth` $= 6$, tested `attention_type` $\in \{$`time`, `freq`, `both`$\}$, and used `num_heads` $\in \{1, 2\}$. In the implementation, `time` attention treats time frames as tokens and frequency bins as per-token features, while `freq` attention uses the transposed view: frequency bins become tokens and time frames become features. The `both` setting runs both transformer branches separately, concatenates their output embeddings, and passes the concatenated representation to the classifier. Because the default KWT depth is 12 [7], the time-only and frequency-only depth-6 variants reduce the parameter count by roughly $2\times$. This stage therefore asks whether a shallower KWT variant can retain or improve the best Stage 2 performance while reducing model size.

4 Results

Stage 1: unknown-handling comparison

Table 4 shows that Strategy A won Stage 1 for all three backbones on validation macro-F1. The margin was modest for BC-ResNet, where Strategy C was close on validation, but the outcome was much clearer for KWT and MatchboxNet.

These results should be interpreted with caution because the strategies did not use the same number of sample draws per epoch. Consequently, training for the same number of epochs produced different numbers of optimizer steps per run. A more fair comparison would fix the total number of training steps instead of the number of epochs per run. Under the executed protocol, however, Strategy D still shows substantial potential: it nearly matches the best Strategy A results despite using roughly $2.5\times$ fewer training steps. Strategy C also produces strong results, but it does so with almost $2\times$ more training steps per run than Strategy A.

Figure 1 shows the full validation threshold sweeps for Strategy D. KWT preferred a middle threshold around $\tau = 0.48$, BC-ResNet around $\tau = 0.44$, and MatchboxNet a noticeably larger $\tau = 0.64$. In all three cases the optimum sat in a relatively narrow plateau.

Stage 2: augmentation study

Since all Stage 1 winners used Strategy A, Stage 2 isolates augmentation effects for that strategy alone. Table 5 summarizes validation macro-F1, test macro-F1, and test accuracy for the full aug-

Table 4: Stage 1 results for all 3×4 backbone-strategy pairs

Backbone	Strategy	Validation macro-F1	Test macro-F1	Test accuracy	Selected τ
BC-ResNet	A	**95.00**	95.64	97.24	-
	B	91.04	91.80	93.98	-
	C	94.75	94.39	96.32	-
	D	93.49	93.41	95.33	0.44
KWT	A	**95.63**	94.59	96.47	-
	B	92.55	91.95	95.09	-
	C	94.78	93.46	96.16	-
	D	94.25	93.21	95.22	0.48
MatchboxNet	A	**95.17**	94.66	96.62	-
	B	92.12	90.99	94.04	-
	C	94.13	92.99	95.88	-
	D	94.22	93.87	95.60	0.64

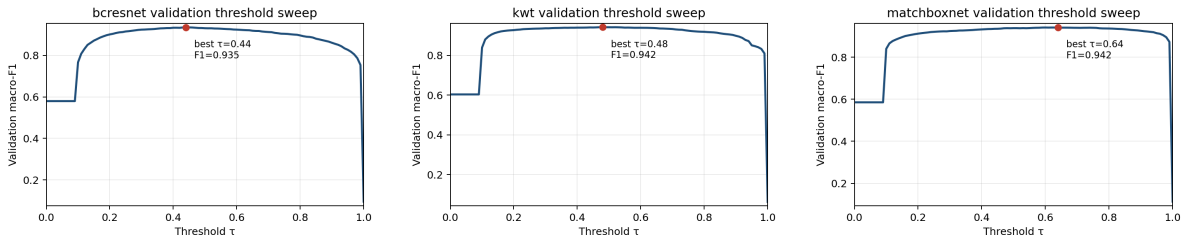

Figure 1: Validation threshold sweeps for Strategy D. From left to right: BC-ResNet, KWT, and MatchboxNet

Table 5: Stage 2 augmentation grid summarized by validation macro-F1, test macro-F1, and test accuracy. Augmentation bits denote (`time shift`, `background noise`, `SpecAugment`)

Backbone	Aug bits	Validation macro-F1	Test macro-F1	Test accuracy
BC-ResNet	000	94.68	94.48	96.56
	001	**95.39**	95.56	**97.28**
	010	94.35	95.21	96.98
	011	94.63	94.67	96.70
	100	95.13	95.00	96.93
	101	95.11	94.53	96.62
	110	95.00	**95.64**	97.24
	111	94.46	95.12	97.11
KWT	000	94.71	93.78	96.28
	001	95.78	95.13	96.94
	010	95.43	94.90	96.83
	011	96.42	**96.50**	97.76
	100	95.33	93.89	96.19
	101	96.03	95.62	97.15
	110	95.63	94.59	96.47
	111	**96.70**	96.45	**97.83**
MatchboxNet	000	94.10	93.39	95.88
	001	95.65	95.64	97.12
	010	94.68	94.59	96.55
	011	95.18	93.62	96.38
	100	94.36	94.69	96.63
	101	**95.73**	**95.68**	**97.31**
	110	95.17	94.66	96.62
	111	95.68	94.59	96.71

mentation grid in one place. The bit order is (`time shift`, `background noise`, `SpecAugment`), where 1 means enabled and 0 means disabled.

For BC-ResNet, Table 5 indicates near-saturation. The best validation run was `aug_001`, which enables only SpecAugment and disables the two waveform-level augmentations, but the best test macro-F1 remained the baseline `aug_110` configuration inherited from Stage 1. BC-ResNet appears robust and does not gain much from heavier augmentation.

KWT shows the largest gains of the entire study. The validation macro-F1 winner `aug_111` improved over the Stage 1 KWT winner by +0.011 macro-F1 on validation and +0.019 on test. The per-class changes explain the gain: the biggest improvements versus Stage 1 were `silence` +0.092, `go` +0.030, `down` +0.021, and `no` +0.016 in test F1. In other words, augmentation mainly fixed the classes that were already the weak points of the plain KWT run.

MatchboxNet improved as well, but with a different preferred augmentation pattern. Table 5 shows that the best run was `aug_101`, which keeps time shift and SpecAugment but removes

background-noise mixing. That configuration improved the Stage 1 winner by +0.006 validation macro-F1 and +0.010 test macro-F1. The mean-factor analysis over the full $2 \times 2 \times 2$ grid points shows that SpecAugment and time shift are helpful on average, whereas background-noise mixing reduces MatchboxNet's mean test macro-F1.

Stage 3: reduced-depth KWT follow-up

Stage 3 tested whether the best Stage 2 KWT setup could be matched by a shallower transformer variant. Table 6 shows a sharp separation between frequency-only attention and the remaining variants. Both frequency-only models dropped to approximately 0.93 test macro-F1, far below the Stage 2 baseline.

The competitive Stage 3 candidates were the time-only and dual-branch models. The best validation Stage 3 run was `attn_time_depth_6_heads_2` with validation macro-F1 = 0.964, but that still remained below the Stage 2 champion's 0.967. The best test Stage 3 run was `attn_both_depth_6_heads_2`, which reached test accuracy = 97.79% and test macro-F1 = 0.966, slightly above the Stage 2 KWT winner in macro-F1 but not in validation evidence. Table 7 compares the three paper KWT variants, to the seven repository KWT models executed in this project.

The executed KWT runs outperformed the paper KWT accuracies despite using a much shorter training schedule: 10,400 steps instead of 23,000 steps with the same batch size 512. The strongest efficiency result is given by the reduced depth-6 time-only variants. They contain 309,791 parameters, which is roughly 2× fewer than the paper KWT-1 model and much fewer than KWT-2 or KWT-3, yet they still reach 97.66–97.69% test accuracy. The Stage 2 depth-12 KWT winner and the Stage 3 dual-branch variants are closer to the paper KWT-1 parameter count, but they also exceed the reported paper accuracies while using less than half as many training steps. The comparison should still be read cautiously because the paper values are multi-run literature summaries whereas the present report is based on single-seed runs.

Table 6: Stage 3 KWT follow-up results with fixed depth = 6

Attention	Heads	Validation macro-F1	Test accuracy [%]	Test macro-F1
both	1	95.98	97.49	96.00
both	2	96.21	**97.79**	**96.57**
freq	1	93.55	95.64	92.80
freq	2	93.38	95.64	92.93
time	1	96.13	97.66	96.47
time	2	**96.44**	97.69	96.33

Table 7: Comparison between the paper KWT family and the seven KWT models executed in this project. Ours models used 10,400 steps with batch size 512, whereas the paper models used 23,000 steps with the same batch size

Source	Model	Dim	MLP-dim	Heads	Layers	# parameters	Test accuracy [%]
Paper	KWT-1	64	256	1	12	607K	97.05 ± 0.23
	KWT-2	128	512	2	12	2,394K	97.36 ± 0.20
	KWT-3	192	768	3	12	5,361K	97.24 ± 0.24
Ours	KWT-1, aug 111	64	256	1	12	608K	97.83
	KWT-1, both d6 h1	64	256	1	6	619K	97.49
	KWT-1, both d6 h2	64	256	2	6	619K	97.79
	KWT-1, freq d6 h1	64	256	1	6	309K	95.64
	KWT-1, freq d6 h2	64	256	2	6	309K	95.64
	KWT-1, time d6 h1	64	256	1	6	309K	97.66
	KWT-1, time d6 h2	64	256	2	6	309K	97.69

Confusion matrix analysis

Figure 2 gives a closer view of the strongest validation model, `kwt_strategy_a_aug_111`. The raw confusion matrix confirms that most residual errors involve only a few labels. On the test set, the largest off-diagonal counts are `unknown`→`silence`, `no`→`unknown`, `right`→`unknown`, and `go`→`no`. The normalized confusion matrix leads to the same conclusion: the model is already excellent on `unknown`, but short and acoustically similar commands remain the limiting cases. Consistent with that picture, the weakest class-wise test F1 values of this run were `no` = 0.939, `go` = 0.942, and `silence` = 0.943.

Figures 3 and 4 show the corresponding confusion matrices for the best Stage 2 configurations of the two convolutional backbones.

For BC-ResNet, `bcresnet_strategy_a_aug_001` the largest remaining issue is `silence`: recall is perfect, but precision is only 0.813, which means that some spoken examples are incorrectly absorbed into the silence class. The weakest class-wise F1 values were `silence` = 0.897, `down` = 0.933, `go` = 0.935, and `no` = 0.939.

For MatchboxNet, `matchboxnet_strategy_a_aug_101` shows a similar row-normalized confusion pattern to the other strong models: `unknown` is handled well, but short commands remain the main source of errors. The weakest class-wise F1 values were `silence` = 0.911, `go` = 0.929, `no` = 0.940, and `right` = 0.949.

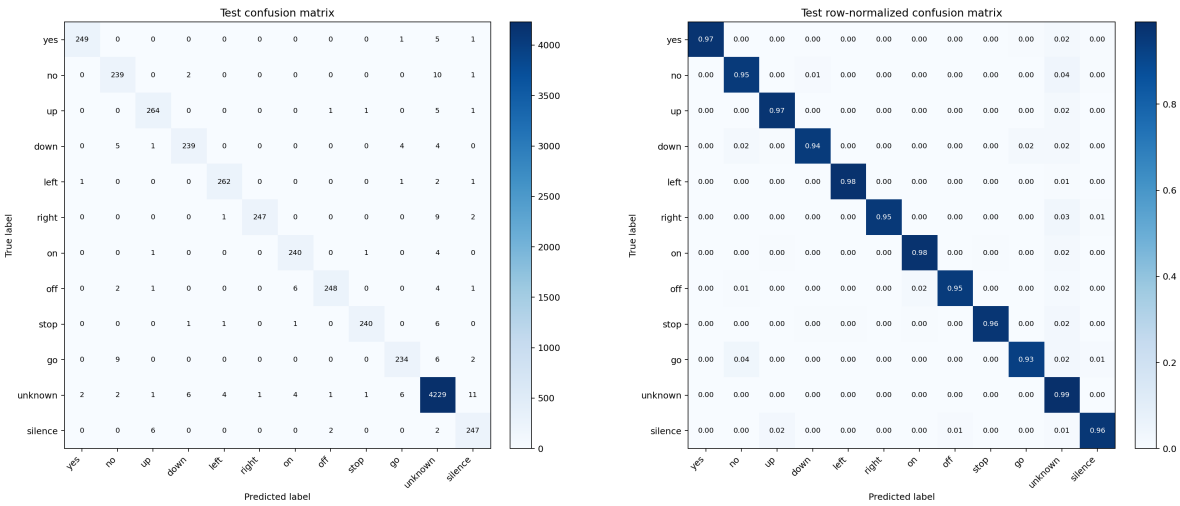

Figure 2: Raw and row-normalized test confusion matrices for `kwt_strategy_a_aug_111`

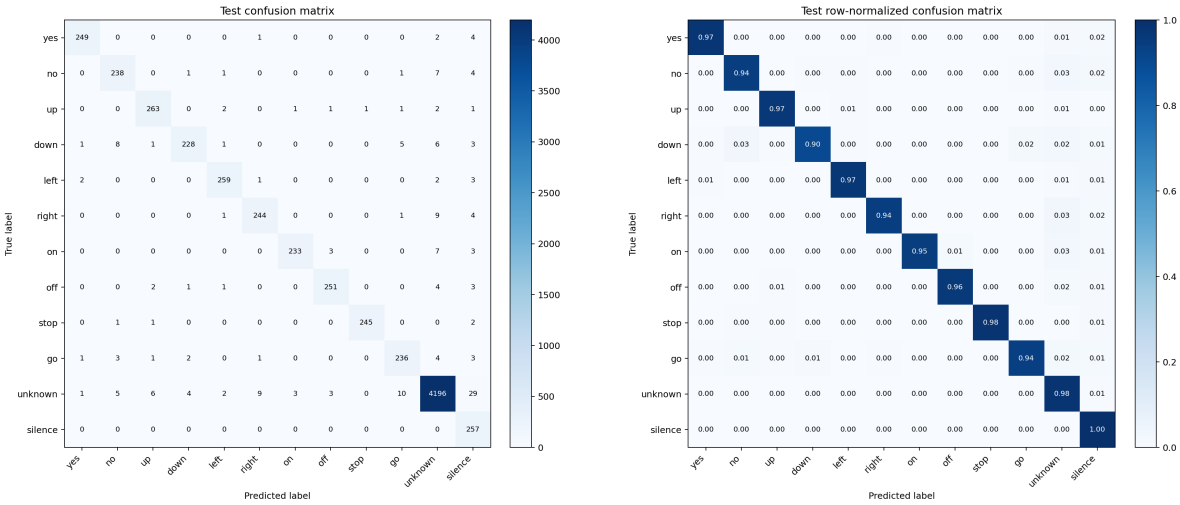

Figure 3: Raw and row-normalized test confusion matrices for `bcresnet_strategy_a_aug_001`

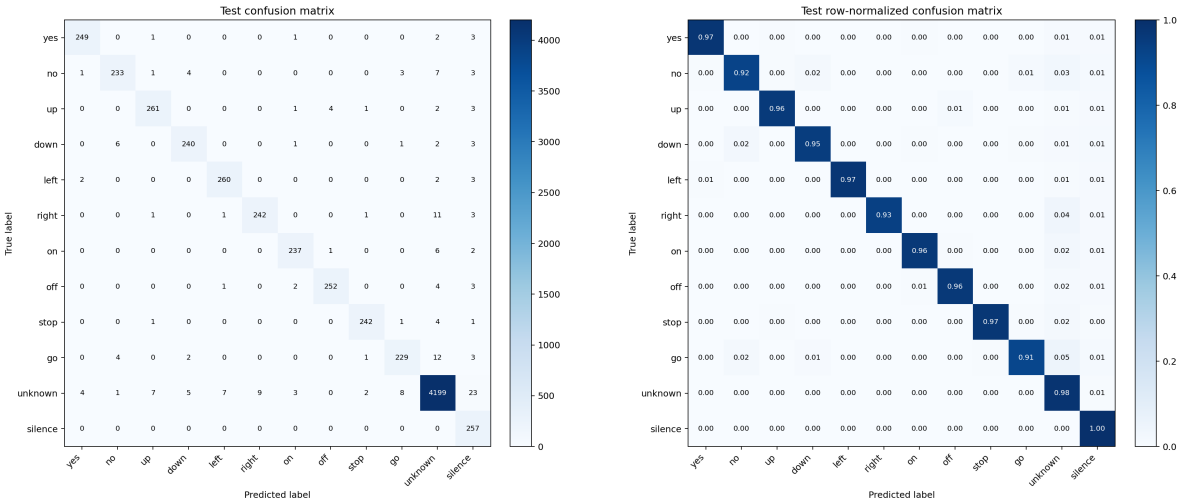

Figure 4: Raw and row-normalized test confusion matrices for the best MatchboxNet Stage 2 run

5 Conclusions, limitations, further work

Conclusions

The main conclusion is that the strongest Stage 1 unknown-handling strategy was the most information-rich formulation rather than the most direct 12-class one. Strategy A consistently achieved the best validation macro-F1 across BC-ResNet, KWT, and MatchboxNet. Strategy B was consistently worst on overall macro-F1. This suggests that direct collapsing into one `unknown` label reduced overall performance through weaker special-class handling.

Augmentation quality depended strongly on the backbone. BC-ResNet was already robust under the Stage 1 baseline and gained little from the Stage 2 grid. MatchboxNet benefited most from time shift and SpecAugment, while background-noise mixing was not clearly helpful for that model. KWT was the clear augmentation winner: enabling all three factors produced the best validation run of the study and improved the weak classes identified for the plain KWT configuration. This supports the interpretation that the transformer backbone could take advantage of stronger regularization instead of overfitting the training set.

Stage 3 does not justify replacing the Stage 2 validation winner. Frequency-only attention was clearly inadequate, while time-only and dual-branch reduced-depth variants remained competitive. The dual-branch two-head variant achieved the best test macro-F1, but its lower validation macro-F1 means it should remain a secondary result under the selection rule. The primary recommendation is therefore `kwt_strategy_a_aug_111`, because it has the best validation evidence and the highest test accuracy. The depth-6 time-only variants are also practically interesting because they use far fewer parameters while retaining strong performance.

Limitations

The most important limitation is statistical. Every configuration was evaluated with a single seed, so the report cannot separate robust performance gaps from seed variance. This matters especially for close comparisons such as the best Stage 2 and Stage 3 KWT runs.

The Stage 1 strategy comparison is also confounded by the sampling regime. Strategy A uses a full shuffled pass over the natural training set, Strategies B and D use weighted sampling with fewer examples per epoch, and Strategy C uses balanced superclass batches with many more draws per epoch. As a result, equal epoch counts do not imply equal optimizer-step budgets or equal data exposure.

The report does not contain latency, memory, throughput, or energy measurements, although these quantities are central for keyword spotting on edge devices. The comparison with published KWT results should also be interpreted cautiously because the literature values are multi-run summaries, whereas our project results are single-run measurements.

Further work

The first follow-up should repeat the strongest Stage 1, Stage 2, and Stage 3 configurations with multiple seeds and report means and standard deviations. A second priority is to re-run the four unknown-handling strategies under matched optimizer-step budgets, so that the effect of the label formulation can be separated from the effect of the sampler.

Although Strategy D did not win Stage 1, its competitive results under a smaller training budget make it a promising candidate for further calibration.

6 Instruction of the application

The repository full experiment pipeline is described in the README. The configuration system is organized around shared defaults and stage-specific experiment files. Common set-

tings are stored under `configs/_shared/`: `configs/_shared/common.yaml` defines dataset paths, output paths, seeds, default augmentation, and evaluation settings, while `configs/_shared/backbones/` contains the backbone-specific defaults. Individual runnable experiments live under `configs/experiments/stage1/`, `configs/experiments/stage2/`, and `configs/experiments/stage3/`.

The manifest files in `configs/manifests/` select which experiment configs are executed. A manifest is a plain text list of config paths, for example `configs/manifests/stage1.txt` points to the Stage 1 experiment files. The Stage 2 and Stage 3 config files are generated from the previous-stage results by `scripts/build_stage2_manifest.py` and `scripts/build_stage3_manifest.py`; these scripts write runnable configs under `configs/experiments/` and update the corresponding manifest with the configs that should be run next. To run only a subset of experiments, edit the relevant manifest so that it contains only the desired config paths.

To reproduce the reported artifacts from scratch, execute the commands below from the repository root after setting up the Python environment and placing the Speech Commands data under `data/audio/`:

```
python scripts/prepare_dataset.py
python scripts/run_manifest.py --manifest configs/manifests/stage1.txt
python scripts/aggregate_results.py
python scripts/build_stage2_manifest.py
python scripts/run_manifest.py --manifest configs/manifests/stage2.txt
python scripts/build_stage3_manifest.py
python scripts/run_manifest.py --manifest configs/manifests/stage3.txt
python scripts/aggregate_results.py
```

The key checkpoints used in this report are:

- `outputs/summaries/stage1/summary.csv` and `outputs/summaries/stage1/winners.csv`,
- `outputs/summaries/stage2/summary.csv`,
- `outputs/summaries/stage3/summary.csv`,
- per-run artifacts under `outputs/runs/{stage}/{experiment_id}/seed_{seed}/`.

Each completed run should contain `val_metrics.json`, `test_metrics.json`, `per_class_metrics.csv`, predictions for validation and test, confusion-matrix plots, a learning-curve plot, and `threshold_sweep.csv` for Strategy D.

References

[1] P. Warden, "Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition," arXiv:1804.03209, 2018.

[2] T. N. Sainath and C. Parada, "Convolutional Neural Networks for Small-footprint Keyword Spotting," in *Proc. Interspeech*, 2015, pp. 1478–1482.

[3] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proc. CVPR*, 2016, pp. 770–778.

[4] F. Chollet, "Xception: Deep Learning with Depthwise Separable Convolutions," in *Proc. CVPR*, 2017, pp. 1251–1258.

[5] B. Kim, S. Chang, J. Lee, and D. Sung, "Broadcasted Residual Learning for Efficient Keyword Spotting," in *Proc. Interspeech*, 2021. https://arxiv.org/abs/2106.04140

[6] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention Is All You Need," in *Proc. NeurIPS*, 2017, pp. 5998–6008.

[7] A. Berg, M. O'Connor, and M. Tairum Cruz, "Keyword Transformer: A Self-Attention Model for Keyword Spotting," in *Proc. Interspeech*, 2021. https://arxiv.org/abs/2104.00769

[8] S. Majumdar and B. Ginsburg, "MatchboxNet: 1D Time-Channel Separable Convolutional Neural Network Architecture for Speech Commands Recognition," in *Proc. Interspeech*, 2020. https://arxiv.org/abs/2004.08531

[9] S. Choi, S. Seo, B. Shin, H. Byun, M. Kersner, B. Kim, D. Kim, and S. Ha, "Temporal Convolution for Real-time Keyword Spotting on Mobile Devices," in *Proc. Interspeech*, 2019, pp. 3372–3376. https://arxiv.org/abs/1904.03814

[10] A. Bendale and T. E. Boult, "Towards Open Set Deep Networks," in *Proc. CVPR*, 2016, pp. 1563–1572.

[11] D. Hendrycks, M. Mazeika, and M. Dietterich, "Deep Anomaly Detection with Outlier Exposure," arXiv:1812.04606, 2018. https://arxiv.org/abs/1812.04606

[12] D. S. Park, W. Chan, Y. Zhang, C.-C. Chiu, B. Zoph, E. D. Cubuk, and Q. V. Le, "SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition," in *Proc. Interspeech*, 2019, pp. 2613–2617.

[13] ARM Software, "keyword-transformer" (official implementation and training scripts). https://github.com/ARM-software/keyword-transformer